Agent-to-Secure Payload Authorization (A2SPA)

Core Protocol Specification

draft-ietf-a2spa-core-protocol-spec-01

Document Version	0.9.1-clean
Status	Working Draft / Core Specification Candidate
Date	June 28, 2026
Working Group	A2SPA Working Group (Open A2SPA WG)
Intended Status	Standards Track
Expires	December 2026

Abstract

This specification defines the Agent-to-Secure Payload Authorization (A2SPA) protocol, a transaction-isolated, runtime authorization framework designed to mitigate blast-radius risks associated with autonomous software systems, model-driven workflows, and automated agents. Unlike identity systems that authenticate an actor or policy engines that evaluate permissions in isolation, A2SPA specifies a deterministic protocol for evaluating and admitting individual execution requests immediately before execution. Traditional access control models rely on static, identity-centric permissions that authorize an agent based on who or what it is. In contrast, A2SPA introduces an execution-time authorization boundary that evaluates exactly what an agent is attempting to execute before it is admitted to the target system.

This document specifies:

1. The logical roles and interaction patterns between the Subject Agent (SA), the Execution Enforcement Point (EEP), and the Authorization Verifier (AV).
2. The mandatory cryptographic baselines, freshness controls, and fail-closed state invariants required to prevent request modification, identity spoofing, and replay attacks.
3. The normative byte-level canonicalization rules necessary to achieve deterministic validation across heterogeneous execution environments.

This specification deliberately does not address upstream model alignment, semantic correctness, or natural language prompt verification. A2SPA focuses exclusively on the deterministic, cryptographic validation of structured execution intents at the boundary of the execution environment.

Document Revision History

Version	Date	Description of Changes	Author/WG
0.1.0	October 2025	Initial architecture definition and core concept drafts.	Open A2SPA WG
0.5.0	January 2026	Separation of EEP and AV roles; addition of Challenge Nonce rules.	Open A2SPA WG
0.9.0	May 2026	Integration of RFC 8785 (JCS) as the mandatory baseline.	Open A2SPA WG
0.9.1-clean	June 2026	Working Draft / Core Spec Candidate. Added invariants, conformance, interoperability, error codes, security considerations, IANA considerations, benchmark methodology, and RFC 6234 reference.	Open A2SPA WG

Implementation Status

In accordance with RFC 7942, this section describes the current deployment and implementation footprint of the A2SPA protocol. This section is informative and does not define conformance.

Current Reference Implementation

The baseline reference implementation (`a2spa-core`) is maintained in Go and Rust. It provides native modules for compiling, signing, parsing, and validating Canonical Execution Requests.

Supported Canonicalization Profiles

- **Baseline JSON Serialization:** RFC 8785 JSON Canonicalization Scheme (JCS).
- **Binary Serialization:** Deterministic CBOR according to RFC 8949 Section 4.2.1, when explicitly negotiated.

Test Vectors

The reference repository should distribute static JSON test vectors containing pre-compiled requests, generated nonces, timestamps, and matching Ed25519 signature outputs. Downstream

implementers should use these vectors to verify signature compatibility across custom language SDKs.

Known Technical Limitations

- **Clock Synchronization Dependencies:** Implementations are sensitive to clock drift under strict TTL constraints.
- **Verification Overhead:** Performance claims should be treated as non-normative unless measured under the benchmark methodology in Section 20.

Table of Contents

1. Introduction
2. Conventions and Terminology
3. Scope
4. Protocol Roles
5. Normative Principle and Protocol Invariants
6. Threat Model
7. Trust and Security Assumptions
8. Architecture Overview
9. Protocol Definition
10. Execution State Machine
11. Cryptographic Requirements
12. Canonicalization Rules
13. Deployment Models
14. Interoperability Considerations
15. Attack and Evidence Matrix
16. Authorization Decision and Error Codes
17. Conformance Requirements
18. Security Considerations
19. IANA Considerations
20. Benchmark Methodology
21. Privacy Considerations
22. The Case Against A2SPA
23. Operational Guidance
24. Terminology and Definitions
25. References

- Appendix A. Design Rationale (Non-Normative)

---

1. Introduction

Autonomous software systems increasingly perform externally observable actions: data modification, API invocation, message transmission, infrastructure configuration, and financial or operational transactions. Existing security systems typically authenticate actors, authorize

broad permissions, secure transport, or record activity after the fact. A2SPA defines a runtime authorization boundary for determining whether a specific structured execution payload should be admitted immediately before execution.

2. Conventions and Terminology

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL in this document are to be interpreted as described in BCP 14, RFC 2119, and RFC 8174 when, and only when, they appear in all capitals.

3. Scope

A2SPA addresses deterministic, cryptographic validation of structured execution intents at the execution boundary. It does not validate natural language prompts, model alignment, semantic correctness, strategy, ethics, legality, or business desirability of an upstream decision.

4. Protocol Roles

- **Subject Agent (SA):** The autonomous software entity that compiles an Execution Request and signs its canonical representation.
- **Execution Enforcement Point (EEP):** The inline enforcement component that intercepts candidate execution, coordinates challenge-response validation, and admits or denies execution.
- **Authorization Verifier (AV):** The independent decision engine that verifies signatures, validates freshness, evaluates policy, and returns an Authorization Decision.

5. Normative Principle and Protocol Invariants

The A2SPA protocol is governed by the following non-negotiable execution boundary:

> **Execution Admission Invariant:** An Execution Request SHALL NOT be admitted to the Execution Environment unless the Execution Enforcement Point has received a successful Authorization Decision from the Authorization Verifier for the corresponding Signed Canonical Execution Request.

Every protocol handshake sequence, state transition, cryptographic configuration, and deployment topology specified hereafter exists to enforce this invariant across runtime boundaries.

5.1 Core Security Goals

A compliant A2SPA implementation SHALL preserve:

1. **Execution Integrity:** The exact payload admitted for execution SHALL be the same payload that was canonicalized, signed, and authorized.
2. **Deterministic Authorization:** Identical Canonical Execution Requests evaluated under identical policy and freshness conditions SHALL produce identical authorization outcomes.
3. **Freshness Protection:** A Signed Canonical Execution Request SHALL be bound to a bounded validity interval and a single-use Challenge Nonce.

4. **Fail-Closed Enforcement:** Absence of valid cryptographic, freshness, canonicalization, or policy evidence SHALL result in denial of execution admission.

5.2 Protocol Invariants

A2SPA implementations MUST enforce:

- **Canonical Representation Invariant:** The signed byte stream MUST be produced from a deterministic canonicalization profile supported by this specification.
- **Authorization Binding Invariant:** The Authorization Decision MUST bind to the specific Canonical Execution Request, Subject Agent identity, Challenge Nonce, policy context, and validity interval.
- **Nonce Finality Invariant:** A Challenge Nonce MUST be accepted at most once. Consumed, expired, or unknown nonces MUST be rejected.
- **EEP Traversal Invariant:** Execution paths that bypass the EEP are non-conformant.
- **Failure Safety Invariant:** Any ambiguity, malformed input, missing verification material, verifier error, timeout, or policy failure MUST result in a fail-closed outcome.

6. Threat Model

6.1 Overview

The A2SPA threat model assumes an environment where execution requests originating from the Subject Agent are untrusted until they have been successfully evaluated by the Authorization Verifier and admitted by the Execution Enforcement Point. The SA may be subject to compromise, software defects, malicious input, configuration errors, or other conditions that result in unauthorized execution requests.

6.2 Security Property Traceability Matrix

Threat	Protected Property	Protocol Control Mechanism	Enforced By
Payload Mutation	Integrity	Section 12 canonicalization bound to cryptographic signatures.	SA generation and AV validation
Replay Attacks	Freshness	Single-use Challenge Nonces and bounded TTL.	EEP generation and AV verification
Delayed Execution with Stale Context	Freshness Protection	Bounded Validity Interval + independent context revalidation.	EEP generation and AV verification
Identity Spoofing	Authenticity	Public-key verification using Ed25519 baseline.	AV cryptographic evaluation

Threat	Protected Property	Protocol Control Mechanism	Enforced By
Parameter Expansion	Authorization	Independent evaluation of parameters against explicit rules.	AV policy engine
Verifier Bypass	Enforcement Integrity	Mandatory EEP traversal and admission control.	EEP structural control
Canonicalization Mismatch	Deterministic Verification	Rejection of duplicate properties, non-UTF-8 streams, or unsupported profiles.	SA and AV normalization

6.3 Adversarial Capabilities and Boundaries

6.3.1 Upstream Agent Compromise

The protocol assumes that the SA decision-making engine may become compromised via runtime exploit, local model exploitation, or configuration error. The attacker may attempt to cause unauthorized actions such as changing a financial recipient address or expanding file access parameters.

A2SPA Mitigation: the signature binds the exact canonical byte stream. Any expansion of parameters requires a signature that matches the modified payload and passes independent AV policy evaluation.

6.3.2 Wire Interception and Modification

An adversary may intercept and modify transit data between the SA and EEP. A2SPA mitigates this by forcing deterministic canonical representation and signature verification. Modifications break signature integrity and result in REJECTED state.

7. Trust and Security Assumptions

7.1 Core Operational Prerequisites

The security guarantees provided by this specification are valid only if the following underlying conditions are maintained by the host environment:

- **Mandatory EEP Traversal:** Network and application topology MUST ensure all execution paths originating from an SA are intercepted by a compliant EEP.
- **Trust Anchor Integrity:** Cryptographic identity roots, public keys, and validation keys MUST be securely managed and remain uncompromised.
- **Time Synchronization:** Participating nodes MUST maintain bounded clock synchronization to evaluate Bounded Validity Intervals reliably.
- **Key Isolation Boundary:** Implementations SHOULD use hardware-backed key storage where available and MUST protect private keys against unauthorized disclosure.

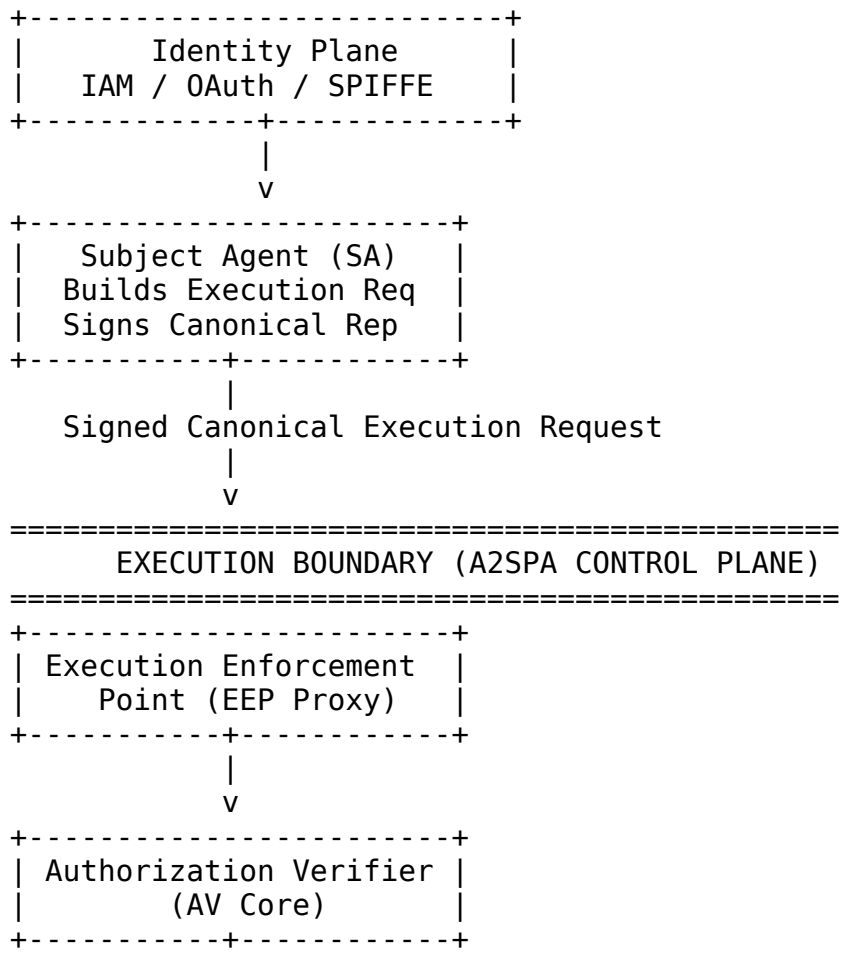
- **Policy Integrity:** Administrators or governance systems MUST ensure AV rules accurately reflect organizational security boundaries.
- **Canonicalization Uniformity:** Participating nodes MUST execute bit-perfect implementations of the baseline serialization profiles without property drift.

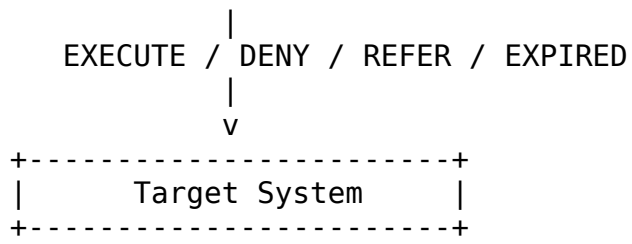
7.2 Trust Profiles

Profile	Description	Minimum Requirement
Development	Local or non-production testing.	Baseline canonicalization and signature validation.
Enterprise	Production business workflows.	Managed keys, audited policies, EEP traversal, HA AV deployment.
Regulated / Critical	Financial, healthcare, government, or safety-impacting workflows.	Hardware-backed keys, strict TTL, immutable audit, documented recovery, independent testing.

8. Architecture Overview

8.1 Topology Diagram





8.2 Component Role Obligations

8.2.1 Subject Agent (SA)

The SA SHALL assemble the parameters of an Execution Request, MUST normalize those parameters into a Canonical Execution Request per Section 12, and MUST sign the transaction payload using its bounded private key to output a Signed Canonical Execution Request.

8.2.2 Execution Enforcement Point (EEP)

The EEP MUST intercept all candidate execution requests, MUST generate and cache a secure single-use Challenge Nonce, and SHALL block request path progression until an explicit successful Authorization Decision is delivered by the AV. The EEP MUST ensure that each Challenge Nonce is accepted at most once.

8.2.3 Authorization Verifier (AV)

The AV MUST verify incoming Signed Canonical Execution Requests against registered public keys, MUST validate that the Challenge Nonce matches active EEP state records, and SHALL output an explicit Authorization Decision when evaluation is complete.

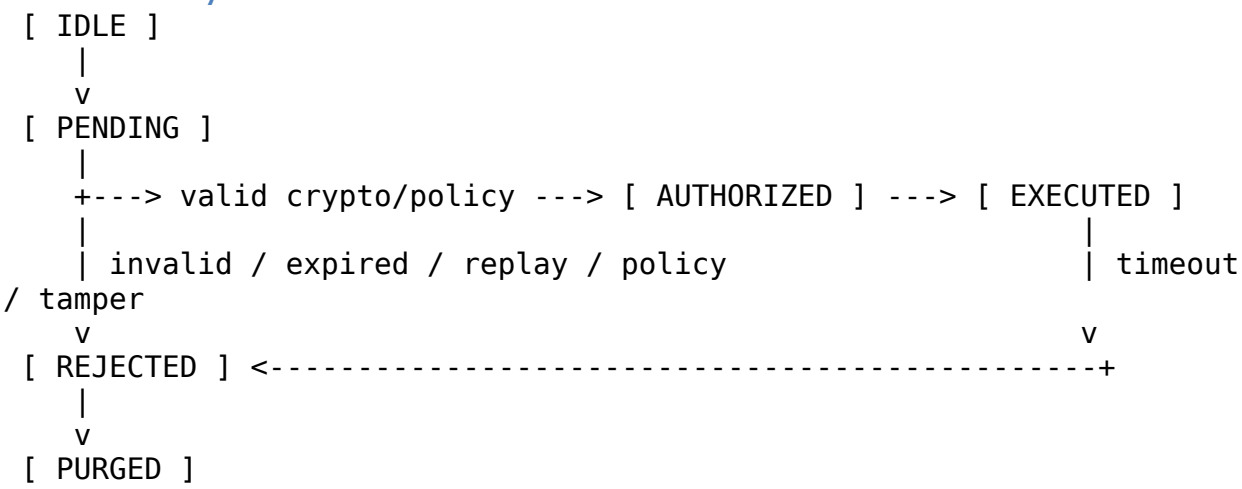
9. Protocol Definition

9.1 Normative Sequence Steps

1. **Intent Initiation:** The SA attempts an action by sending an unsigned Execution Request to the EEP.
2. **Challenge Generation:** The EEP intercepts the request, blocks execution entry, generates a transactional Challenge Nonce, and transmits it back to the SA.
3. **Payload Signing:** The SA binds the Challenge Nonce to the parameters, generates a Canonical Execution Request, signs the byte stream, and transmits the Signed Canonical Execution Request to the EEP.
4. **Verification Routing:** The EEP forwards the Signed Canonical Execution Request and challenge context to the AV.
5. **Authorization Evaluation:** The AV evaluates signatures, freshness, canonicalization, and policy, then returns an Authorization Decision to the EEP.
6. **Enforcement and Admission:** The EEP MUST invalidate the Challenge Nonce. If the decision is EXECUTE, the EEP admits the request into the Execution Environment.

10. Execution State Machine

10.1 State Lifecycles



10.2 State Definitions

- **IDLE:** Default state. No active execution requests.
- **PENDING:** Intent has been intercepted by the EEP and is undergoing challenge binding or verification.
- **AUTHORIZED:** The AV has validated the request and generated an ephemeral execution token.
- **EXECUTED:** Terminal success state. The EEP permitted the action and logged proof.
- **REJECTED:** Terminal failure state triggered by signature mismatch, malformed canonicalization, expired nonce, replay, or policy violation.
- **PURGED:** Post-terminal cleanup state. Ephemeral challenge data and sensitive buffers are cleared according to implementation policy.

11. Cryptographic Requirements

11.1 Primitives and Enforcement Obligations

- **Mandatory Signature Baseline:** All compliant implementations MUST support Ed25519 as defined in RFC 8032 as the default signature validation baseline.
- **Enterprise Compatibility Profile:** Implementations MAY support ECDSA P-256 with SHA-256 as defined in FIPS 186-5. If ECDSA is supported, the AV MUST protect against signature malleability by enforcing low-S value constraints.
- **Hashing Functions:** The default hashing function for digesting payloads prior to signing MUST be SHA-256 as defined in RFC 6234. Implementations MAY support SHA-512.
- **Freshness Invariants:** The EEP MUST generate a cryptographically secure Challenge Nonce for every transaction handshake and MUST ensure that each Challenge Nonce is accepted at most once.
- **Key Isolation Boundary:** The SA MUST NOT hold or manage cryptographic keys capable of expanding, altering, or elevating its own operational authority. The SA key

proves authenticity and integrity; authorization properties remain the responsibility of the AV.

- **Fail-Closed Mandate:** On invalid signature, expired TTL, replayed or missing nonce, malformed canonicalization, AV timeout, or policy failure, the EEP MUST halt execution and transition the request to REJECTED.

12. Canonicalization Rules

12.1 Mandatory Baseline Profile: RFC 8785 (JCS)

A2SPA-compliant implementations MUST support the JSON Canonicalization Scheme (JCS) as defined in RFC 8785 as the default serialization profile. Implementations MUST NOT claim baseline interoperability or conformance unless RFC 8785 verification is fully supported.

The normalization engine MUST sort all JSON keys lexicographically by code point, MUST strip whitespace outside string literals, and SHALL reject property duplications prior to signature calculation.

12.2 High-Performance Profile: Deterministic CBOR

Implementations MAY support deterministic CBOR as defined in RFC 8949 Section 4.2.1. This profile MUST be explicitly negotiated between the SA and EEP. If negotiation fails or is unsupported, the system MUST fall back to RFC 8785 baseline.

12.3 Internal SDK Profile: Protocol Buffers

Protocol Buffers MAY be used strictly for internal, intra-service SDK communication within a closed deployment boundary. Protobuf MUST NOT be exposed as a public edge-interoperability profile unless constrained by a deterministic, version-stable serialization profile outside the baseline of this specification.

12.4 Custom Formats

Implementations MUST NOT utilize proprietary or custom serialization formats for baseline signature verification.

12.5 Canonicalization Invariants

- **No Property Duplication:** Payloads containing duplicate JSON keys MUST be rejected before canonicalization.
- **String Encoding:** All payload representations MUST be encoded strictly as UTF-8 octet streams.
- **Character Escaping:** Only characters required to be escaped by the baseline specification may carry escape sequences.

13. Deployment Models

13.1 Normative Architectural Conformance

The deployment profiles described in this section are informative. Conformance is determined by core protocol behavior and invariant satisfaction, not deployment topology. The A2SPA protocol defines logical roles, not physical components or strict network boundaries.

13.2 Deployment Profile Taxonomy

Profile	Assurance Level	Application Modification	Target Workload Fit
Gateway Profile	Medium Assurance	None	Legacy APIs, third-party SaaS integrations
Sidecar Profile	High Assurance	Minimal	Containerized microservices, Kubernetes
Service Mesh Profile	High Assurance	Low	Cloud-native platform infrastructures
Embedded Runtime Profile	Very High Assurance	Moderate	Agent frameworks, autonomous workflows
Target-Native Profile	Maximum Assurance	High	Databases, payment rails, ERP systems

13.3 Profile Specifications

13.3.1 Gateway Profile

- **Topology:** EEP as inline API gateway filter or plugin; AV behind gateway.
- **Advantages:** Fast adoption without upstream or downstream code modification.
- **Operational Trade-offs:** Dependency on centralized ingress path.
- **Residual Risks:** Perimeter-only defense if internal routes bypass the gateway.

13.3.2 Sidecar Profile

- **Topology:** EEP as container-isolated proxy beside the SA.
- **Advantages:** Localized interception of outbound requests.
- **Operational Trade-offs:** Additional memory and CPU footprint.
- **Residual Risks:** Shared-kernel and container escape risks remain.

13.3.3 Target-Native Enforcement Profile

- **Topology:** EEP embedded in target system such as database trigger, ERP transaction hook, or payment rail validation module.
- **Advantages:** No downstream execution space for unverified requests.
- **Operational Trade-offs:** Intrusive integration engineering.

- **Residual Risks:** Critical systems fail closed if AV availability is not engineered appropriately.

14. Interoperability Considerations

A2SPA is transport-independent. The protocol defines logical authorization roles and verification requirements, not a mandatory transport substrate.

Compliant implementations MAY expose A2SPA verification over HTTPS, REST, gRPC, message queues, service mesh filters, API gateway plug-ins, workflow engines, MCP-style tool invocation layers, A2A-style agent communication layers, or target-native transaction hooks, provided all protocol invariants remain satisfied.

Transport authentication, workload identity, TLS, OAuth, IAM, or policy approval MUST NOT be treated as a substitute for A2SPA execution-time authorization.

A2SPA is designed to complement identity, policy, and transport-security systems. It does not replace IAM, OAuth, SPIFFE/SPIRE, mTLS, OPA, SIEM, or runtime monitoring systems.

15. Attack and Evidence Matrix

15.1 Security Objectives

1. **Authenticity:** The execution request originated from a recognized, cryptographically bound SA.
2. **Integrity:** The canonical request has not been altered, mutated, or truncated post-signing.
3. **Authorization:** The requested parameters are explicitly permitted under current AV policy invariants.
4. **Freshness:** The request is bound to a single transaction window.
5. **Deterministic Verification:** Identical canonical requests under identical conditions produce identical outcomes.
6. **Fail-Closed Enforcement:** Absence of evidence results in denial of execution admission.

15.2 Evidence Matrix

Threat	Security Property	Protocol Mechanism	Result if Mechanism Holds	Out of Scope / Environmental Assumption
Payload Mutation	Integrity	RFC 8785 JCS + signature verification	Modified payload fails validation and enters REJECTED.	Upstream compromise of signing authority.
Replay Attacks	Freshness	EEP nonce + TTL	Replay fails nonce or TTL check.	EEP nonce cache compromise.
Delayed	Freshness	Bounded	Execution	Authorization

Threat	Security Property	Protocol Mechanism	Result if Mechanism Holds	Out of Scope / Environmental Assumption
Execution with Stale Context	Protection	Validity Interval enforced independently of nonce reuse	outside the approved authorization window is denied even if the nonce is still technically unconsumed.	context (permissions, environment) drifting within an otherwise-valid TTL window.
Identity Spoofing	Authenticity	Ed25519 verification	Forged or unmapped requests fail.	Private key theft.
Parameter Expansion	Authorization	AV policy evaluation	Out-of-bounds parameters rejected.	Logical policy flaws.
Verifier Bypass	Enforcement Integrity	Mandatory EEP invariant	Payload lacking AV grant rejected.	Misconfigured topology.
Canonicalization Mismatch	Deterministic Verification	Strict RFC 8785 normalization	Serialization variations cause mismatch.	Non-compliant serialization engine.

16. Authorization Decision and Error Codes

The Authorization Verifier SHALL return an explicit Authorization Decision to the Execution Enforcement Point.

16.1 Decision Classes

- **EXECUTE:** The request satisfies cryptographic, freshness, canonicalization, and policy requirements.
- **DENY:** The request fails one or more mandatory checks.
- **REFER:** The request requires human, administrative, or higher-order policy review before execution.
- **EXPIRED:** The request is outside its validity interval or uses an expired Challenge Nonce.

16.2 Canonical Error Codes

Code	Status	Meaning
1000	EXECUTE	Request authorized for execution.
2001	DENY	Invalid signature.
2002	DENY	Unknown Subject Agent

Code	Status	Meaning
		key.
2003	DENY	Canonicalization failure.
2004	DENY	Duplicate JSON property detected.
2005	DENY	Non-UTF-8 payload.
2006	DENY	Policy violation.
2007	DENY	Challenge Nonce missing.
2008	DENY	Challenge Nonce replay detected.
2009	EXPIRED	TTL expired.
2010	DENY	Unsupported canonicalization profile.
2011	DENY	Unsupported signature algorithm.
2012	REFER	Human approval required.
2013	DENY	EEP state mismatch.
2014	DENY	AV unavailable and fail-closed enforcement triggered.

The EEP MUST treat any unknown, malformed, missing, or ambiguous Authorization Decision as DENY.

17. Conformance Requirements

An implementation conforms to this specification only if it satisfies all mandatory requirements in this section.

A compliant implementation MUST:

1. Support RFC 8785 JCS as the baseline canonicalization profile.
2. Support Ed25519 signature verification as defined in RFC 8032.
3. Enforce mandatory EEP traversal before execution admission.
4. Generate, validate, consume, and reject Challenge Nonces according to the Nonce Finality Invariant.
5. Bind authorization decisions to the exact Canonical Execution Request.
6. Enforce bounded validity intervals.
7. Reject malformed, ambiguous, duplicate-property, non-UTF-8, or unsupported canonicalization inputs.

8. Fail closed on verifier timeout, cryptographic failure, canonicalization failure, nonce failure, policy denial, or missing authorization evidence.
9. Emit auditable security events for denied, expired, referred, and executed requests.
10. Prevent successful execution of any request lacking a valid Authorization Decision.

An implementation MUST NOT claim conformance if it allows execution based only on identity authentication, transport security, static IAM permission, logging, or post-execution audit evidence.

18. Security Considerations

A2SPA reduces execution-time blast-radius risk by requiring deterministic authorization of structured execution payloads before admission to a target system. Its security guarantees depend on correct implementation and deployment of the following controls.

18.1 Key Management

Private keys used by Subject Agents MUST be protected against disclosure. Implementations SHOULD use hardware-backed key storage where available. Key rotation, revocation, and trust-anchor distribution MUST be handled through secure operational procedures.

18.2 Clock Synchronization

Bounded validity intervals depend on reliable time synchronization. Implementations MUST define maximum tolerated clock skew. Deployments with unstable clock synchronization SHOULD use conservative TTL windows or additional freshness validation.

18.3 Denial of Service

A2SPA introduces verification work at the execution boundary. Implementations SHOULD rate-limit malformed requests, replay attempts, invalid signatures, and repeated nonce failures. EEP and AV components SHOULD be horizontally scalable where high-throughput workloads are expected.

18.4 Canonicalization Attacks

Implementations MUST reject duplicate properties, unsupported encodings, non-UTF-8 streams, ambiguous number representations, unsupported serialization profiles, and any input that cannot be deterministically canonicalized.

18.5 Downgrade Attacks

Implementations MUST NOT downgrade from the baseline RFC 8785 profile to a weaker or proprietary serialization format. Optional profiles, including deterministic CBOR, MUST be explicitly negotiated and verified.

18.6 Verifier Availability

If the AV is unavailable, unreachable, or returns an invalid response, the EEP MUST fail closed. Availability engineering MUST NOT weaken the Execution Admission Invariant.

18.7 Out-of-Scope Risks

A2SPA does not determine whether an upstream model intent is semantically correct, ethical, financially optimal, legally appropriate, or strategically desirable. A valid request can still represent a bad decision if the upstream system produced an authorized but mistaken intent.

19. IANA Considerations

This specification requires no IANA actions.

20. Benchmark Methodology

Performance claims SHOULD NOT be treated as normative unless measured under a published benchmark methodology. Benchmark reports SHOULD include:

- hardware model
- CPU architecture
- operating system
- compiler version
- runtime version
- cryptographic library version
- canonicalization profile
- request size
- concurrency level
- sample size
- median latency
- p95 latency
- p99 latency
- error rate
- clock synchronization assumptions
- network topology
- whether EEP and AV are co-located or remote

Implementers SHOULD publish raw test vectors and reproducible benchmark harnesses when making public performance claims.

21. Privacy Considerations

A2SPA authorization receipts and audit logs can contain metadata about agents, targets, actions, timestamps, policies, and execution outcomes. Implementations SHOULD minimize sensitive payload retention, avoid unnecessary storage of raw execution content, and support retention policies appropriate to the deployment environment.

22. The Case Against A2SPA

22.1 Functional Equivalence via Existing Compositions

Skeptics may accurately observe that the cryptographic and architectural building blocks of A2SPA are not novel in isolation. A deployment that manually combines robust workload identity, application-layer request signing, deterministic canonicalization profiles, structured policy evaluation, replay protection, and mandatory inline execution enforcement can achieve security properties functionally equivalent to those described by this specification without adopting a novel protocol label.

The purpose of A2SPA is not to assert that these security capabilities cannot be composed elsewhere. Rather, it specifies a common, transaction-isolated execution-time authorization model intended to facilitate interoperable implementations and independent validation across heterogeneous autonomous systems.

22.2 The Autonomy Decoupling Problem (Intent vs. Correctness)

A2SPA evaluates whether a candidate execution request satisfies cryptographic and enterprise authorization requirements. It does not determine whether the requested operational objective is correct, accurate, ethical, financially appropriate, or strategically desirable.

If an upstream SA produces a mistaken but authenticated, fresh, and policy-compliant canonical payload, A2SPA will admit the operation. A2SPA enforces the security perimeter at the execution edge; it does not solve upstream alignment or correctness challenges.

22.3 Compromised Trust Roots and Cascading Failures

Like all architectures reliant on public-key infrastructure, the security guarantees of A2SPA depend on the integrity of cryptographic dependencies. If private keys, HSMs, AV signing infrastructure, or EEP trust anchors are compromised, authenticity and integrity guarantees can fail globally.

22.4 Explicit Architectural Assumptions

The architectural assumptions underlying A2SPA are defined normatively in Section 7. This section does not restate those assumptions. Failure to maintain those assumptions can weaken or invalidate the security guarantees described by this specification.

22.5 Conditions for Falsification

The foundational architectural thesis of A2SPA is presented as a technical hypothesis subject to empirical engineering validation. This thesis would be weakened or invalidated if independent research demonstrates that:

1. Existing standard enterprise access control mechanisms provide identical security properties under identical autonomous agent threat models without architectural modification.
2. Deterministic runtime payload verification can be achieved reliably without a distinct, transaction-isolated execution-time authorization layer.

3. The operational complexity and latency overhead introduced by the SA-EEP-AV handshake structurally outweigh measurable reduction in blast-radius risk within enterprise environments.

23. Operational Guidance

- **Audit Receipts:** Implementations SHOULD emit durable records for EXECUTE, DENY, REFER, and EXPIRED decisions.
- **Policy Change Control:** Policy updates SHOULD be versioned and auditable.
- **Incident Response:** Repeated invalid signatures, nonce replays, or canonicalization failures SHOULD be treated as high-priority security events.
- **Human Approval:** REFER workflows SHOULD bind approvals to the exact canonical request, not to a natural-language summary alone.

24. Terminology and Definitions

24.1 Scope

Unless otherwise specified, the terminology defined in this section is normative. Capitalized terms have the meanings assigned in this section and SHALL be interpreted consistently across all sections of this specification.

24.2 Definitions

- **Subject Agent (SA):** The autonomous software entity, model-driven workflow, or system process that generates operational intents and compiles them into a candidate request.
- **Execution Enforcement Point (EEP):** The inline protocol enforcement component responsible for intercepting candidate execution requests, coordinating challenge-response validation, enforcing AV decisions, and determining whether an operation is admitted.
- **Authorization Verifier (AV):** The independent decision engine responsible for verifying signatures, validating freshness, and evaluating payload parameters against policy invariants.
- **Execution Request:** The raw representation of a proposed operation compiled by an SA.
- **Canonical Execution Request:** The immutable, bit-perfect serialization of an Execution Request generated in accordance with a specified profile.
- **Signed Canonical Execution Request:** A Canonical Execution Request cryptographically signed by the private key of an SA.
- **Challenge Nonce:** A cryptographically secure, single-use token generated transactionally by the EEP to guarantee freshness and prevent replay.
- **Authorization Decision:** The deterministic status indicator issued by the AV to the EEP upon evaluating a Canonical Execution Request.
- **Execution Token:** An ephemeral, short-lived cryptographic grant issued alongside a successful Authorization Decision.

- **Execution:** The irreversible commitment of an operation causing externally observable state change.
- **Execution Environment:** The software and hardware components responsible for committing an Execution Request to a target system after admission by the EEP.

25. References

25.1 Normative References

- **[RFC2119]** Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, https://www.rfc-editor.org/info/rfc2119.
- **[RFC8174]** Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, https://www.rfc-editor.org/info/rfc8174.
- **[RFC8032]** Josefsson, S. and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (Ed25519)", RFC 8032, DOI 10.17487/RFC8032, January 2017, https://www.rfc-editor.org/info/rfc8032.
- **[RFC8785]** Rundgren, A., Leonard, P., and F. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, https://www.rfc-editor.org/info/rfc8785.
- **[RFC8949]** Bormann, C. and P. Hoffman, "Concise Binary Object Representation (CBOR)", STD 94, RFC 8949, DOI 10.17487/RFC8949, December 2020, https://www.rfc-editor.org/info/rfc8949.
- **[RFC6234]** Eastlake 3rd, D. and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)", RFC 6234, DOI 10.17487/RFC6234, May 2011, https://www.rfc-editor.org/info/rfc6234.
- **[RFC7942]** Sheffer, Y. and A. Farrel, "Improving Awareness of Running Code: The Implementation Status Section", BCP 205, RFC 7942, DOI 10.17487/RFC7942, July 2016, https://www.rfc-editor.org/info/rfc7942.
- **[FIPS1865]** National Institute of Standards and Technology, "Digital Signature Standard (DSS)", FIPS 186-5, DOI 10.6028/NIST.FIPS.186-5, February 2023, https://csrc.nist.gov/pubs/fips/186-5/final.

Appendix A. Design Rationale (Non-Normative)

A.1 Selection of Cryptographic Primitives

The protocol specifies Ed25519 as its mandatory signature baseline to optimize performance and resilience at the execution edge. Ed25519 signatures are compact and avoid per-message randomness during signing, reducing implementation risk within constrained runtimes.

A.2 Decentralization of Freshness

A2SPA routes Challenge Nonce generation to the EEP because the EEP controls the execution boundary and has immediate state awareness of a single transaction loop.

A.3 Universal Baseline Serialization

A2SPA requires RFC 8785 JCS to eliminate whitespace, property ordering, and representation ambiguity at the signature boundary.

A.4 Architectural Decoupling of EEP and AV

Separating enforcement from decision logic keeps the EEP minimal while allowing the AV to perform deeper policy evaluation and contextual checks.
